

Algoritmos de optimización - Seminario

Nombre y Apellidos:

- Bhuvana Rangaiah Cheemala Marri
- Juan Pablo Salas Suarez

Url: <https://github.com/bhuvanarangaiah/03MAIR-Algoritmos-de-Optimizacion-2026>

Problema:

1. Sesiones de doblaje

Descripción del problema:

- Se precisa coordinar el doblaje de una película. Los actores del doblaje deben coincidir en las tomas en las que sus personajes aparecen juntos en las diferentes tomas. Los actores de doblaje cobran todos la misma cantidad por cada día que deben desplazarse hasta el estudio de grabación independientemente del número de tomas que se graben. No es posible grabar más de 6 tomas por día. El objetivo es planificar las sesiones por día de manera que el gasto por los servicios de los actores de doblaje sea el menor posible.

- Los datos son:
 - Número de actores: 10
 - Número de tomas : 30

Actores/Tomas : <https://bit.ly/36D8luK>

- 1 indica que el actor participa en la toma
- 0 en caso contrario

....

(*) La respuesta es obligatoria

Carga de Datos:

```
In [1]: # =====
# CARGA DE DATOS - Problema 1: Sesiones de doblaje
# =====

"""
Datos: 30 tomas x 10 actores. M[t][a] = 1 si el actor a participa en la toma t, 0 en caso contrario.
Se cargan los datos directamente a continuación para que el notebook no depende de ficheros externos.
"""

MAX_TOMAS_POR_DIA = 6

# Matriz de participacion (formato de entrada, tal cual la hoja)
matriz = [
    [1,1,1,1,1,0,0,0,0,0], # toma 1
    [0,0,1,1,1,0,0,0,0,0], # toma 2
    [0,1,0,0,1,0,1,0,0,0], # toma 3
    [1,1,0,0,0,0,1,1,0,0], # toma 4
    [0,1,0,1,0,0,0,1,0,0], # toma 5
    [1,1,0,1,1,0,0,0,0,0], # toma 6
    [1,1,0,1,1,0,0,0,0,0], # toma 7
    [1,1,0,0,0,1,0,0,0,0], # toma 8
    [1,1,0,1,0,0,0,0,0,0], # toma 9
    [1,1,0,0,0,1,0,0,1,0], # toma 10
    [1,1,1,0,1,0,0,1,0,0], # toma 11
    [1,1,1,1,0,1,0,0,0,0], # toma 12
    [1,0,0,1,1,0,0,0,0,0], # toma 13
    [1,0,1,0,0,1,0,0,0,0], # toma 14
    [1,1,0,0,0,0,1,0,0,0], # toma 15
    [0,0,0,1,0,0,0,0,0,1], # toma 16
    [1,0,1,0,0,0,0,0,0,0], # toma 17
    [0,0,1,0,0,1,0,0,0,0], # toma 18
    [1,0,1,0,0,0,0,0,0,0], # toma 19
    [1,0,1,1,1,0,0,0,0,0], # toma 20
    [0,0,0,0,0,1,0,1,0,0], # toma 21
    [1,1,1,1,0,0,0,0,0,0], # toma 22
    [1,0,1,0,0,0,0,0,0,0], # toma 23
    [0,0,1,0,0,1,0,0,0,0], # toma 24
    [1,1,0,1,0,0,0,0,0,1], # toma 25
    [1,0,1,0,1,0,0,0,1,0], # toma 26
    [0,0,0,1,1,0,0,0,0,0], # toma 27
    [1,0,0,1,0,0,0,0,0,0], # toma 28
    [1,0,0,0,1,1,0,0,0,0], # toma 29
    [1,0,0,1,0,0,0,0,0,0], # toma 30
]
```

```
N_ACTORES = len(matriz[0])          # 10

# Transformamos la matriz a lista de conjuntos de actores
tomas_completo = []
for fila in matriz:
    actores_de_toma = set()
    for a in range(N_ACTORES):
        if fila[a] == 1:
            actores_de_toma.add(a + 1)
    tomas_completo.append(actores_de_toma)

# AJUSTE para probar el modelo: Seleccionamos solo las primeras N tomas dado que como se indicó en los puntos
anteriores no es posible probar con 30
NUM_TOMAS_PRUEBA = 10

tomas_prueba = tomas_completo[:NUM_TOMAS_PRUEBA]
tomas = [ {a+1 for a in range(N_ACTORES) if fila[a]==1} for fila in matriz ]

N_TOMAS = len(tomas_prueba)

print('Actores por cada toma: ', tomas)
```

Actores por cada toma: [{1, 2, 3, 4, 5}, {3, 4, 5}, {2, 5, 7}, {8, 1, 2, 7}, {8, 2, 4}, {1, 2, 4, 5}, {1, 2, 4, 5}, {1, 2, 6}, {1, 2, 4}, {1, 2, 6, 9}, {1, 2, 3, 5, 8}, {1, 2, 3, 4, 6}, {1, 4, 5}, {1, 3, 6}, {1, 2, 7}, {10, 4}, {1, 3}, {3, 6}, {1, 3}, {1, 3, 4, 5}, {8, 6}, {1, 2, 3, 4}, {1, 3}, {3, 6}, {1, 2, 10, 4}, {1, 3, 5, 9}, {4, 5}, {1, 4}, {1, 5, 6}, {1, 4}]

(*)¿Cuántas posibilidades hay sin tener en cuenta las restricciones?

¿Cuántas posibilidades hay teniendo en cuenta todas las restricciones.

Respuesta

Sin Restricciones:

Al no tener ninguna restricción, y si se tienen etiquetados los días de grabación (día 1, día 2, día 3, etc) el número de posibilidades se reduce a un problema de combinatoria. Cada una de las 30 tomas podría asignarse a cualquiera de los días disponibles. Si no se fija un máximo de días, en teoría se podría usar hasta 30 días (una toma por día), por lo tanto, para X días posibles cada toma tiene X opciones (que rueda el día 1, o el día 2, o el día 3..., o el día X). En conclusión, se tiene X^30 posibilidades

Ahora bien, si los días no tienen nombre (día 1, día 2, día 3) sino que solo interesa agrupar las tomas en un mismo bloque de trabajo, el resultado se puede obtener usando el NUMERO DE BELL B(30), que se calcula con el "triangulo de Bell" (cada fila empieza con el ultimo valor de la anterior; cada entrada = izquierda + diagonal superior-izq; el 1er valor de cada fila es el numero de Bell).

Con Restricciones:

Si se distribuyen 30 tomas con máximo 6 tomas/día, se tendrían exactamente 5 días para realizar todas las tomas. En este caso la forma de repartir las 30 tomas en 5 grupos de 6 sería: 30! / ((6!)^5 * 5!)

```
In [2]: # =====
# NUMERO DE POSIBILIDADES - Problema 1: Sesiones de doblaje
# =====
import math

# ---- SIN restricciones -----
""" Al no tener ninguna restricción, y si se tienen etiquetados los días de grabación (día 1, día 2, día 3, etc) el
número de posibilidades
se reduce a un problema de combinatoria. Cada una de las 30 tomas podría asignarse a cualquiera de los días
disponibles.
Si no se fija un máximo de días, en teoría se podría usar hasta 30 días (una toma por día), por lo tanto, para X dias
posibles cada toma tiene X
opciones (que rueda el día 1, o el día 2, o el día 3...., o el día X). En conclusión, se tiene X^30 posibilidades
"""

def posibilidades(X, tomas=30):
    return X**tomas
print("SIN RESTRICCIONES")
for X in [2, 10, 30]:
    print(f"Número de posibilidades si se rueda en {X} días: {posibilidades(X),}")

"""
Ahora bien, si los días no tienen nombre (día 1, día 2, día 3) sino que solo interesa agrupar las tomas en un mismo
bloque de trabajo, el resultado
se puede obtener usando el NUMERO DE BELL B(30), que se calcula con el "triangulo de Bell" (cada fila empieza con el
ultimo valor de la anterior;
cada entrada = izquierda + diagonal superior-izq; el 1er valor de cada fila es el numero de Bell).
"""
def bell(n):
    fila = [1]          # B(0) = 1
    for _ in range(n):
        siguiente = [fila[-1]] # empieza con el ultimo de la fila previa
```

```

        for x in fila:
            siguiente.append(siguiente[-1] + x)
        fila = siguiente
    return fila[0]                # primer elemento = B(n)

B30 = bell(30)

# ---- CON restricciones -----
""" Si se distribuyen 30 tomas con máximo 6 tomas/día, se tendrían exactamente 5 días para realizar todas las tomas.
En este caso la forma de repartir las 30 tomas en 5 grupos de 6 sería: 30! / ( (6!)^5 * 5! ) """

con_restr = math.factorial(30) // (math.factorial(6)**5 * math.factorial(5))

print()
print(f"SIN restricciones ni etiquetas de días -> B(30): {B30:,} posibilidades")
print("                                = {:.3e}".format(B30))
print()
print(f"CON restriccion (5 días x 6 tomas): 30! / (6!^5 * 5!)= {con_restr:,} posibilidades")
print("                                = {:.3e}".format(con_restr))

```

SIN RESTRICCIONES

Número de posibilidades si se rueda en 2 días: 1,073,741,824

Número de posibilidades si se rueda en 10 días: 1,000,000,000,000,000,000,000,000,000,000,000

Número de posibilidades si se rueda en 30 días: 205,891,132,094,649,000,000,000,000,000,000,000,000,000

SIN restricciones ni etiquetas de días -> B(30): 846,749,014,511,809,332,450,147 posibilidades
= 8.467e+23

CON restriccion (5 días x 6 tomas): 30! / (6!^5 * 5!)= 11,423,951,396,577,720 posibilidades
= 1.142e+16

Modelo para el espacio de soluciones

(*) ¿Cual es la estructura de datos que mejor se adapta al problema? Argumentalo.(Es posible que hayas elegido una al principio y veas la necesidad de cambiar, argumentalo)

Respuesta

Para este tipo de problema de optimización conviene distinguir dos estructuras de datos con tipologías distintas:

A) Datos de entrada (fijos): quien participa en cada toma. Si se pretende resolver este problema con algoritmos de búsqueda como backtracking o cualquier otro, la mejor estructura de entrada sería inicialmente como una **matriz** 30×10 de 0 y 1. Es cómoda para leer y verificar y nos sirve para validar quien participa en cada toma o cuantas veces aparece un actor sin embargo plantear una solución desde la matriz puede resultar ineficiente y costoso computacionalmente. Por eso convertimos cada toma en un **conjunto (set) de actores** permitiendo que directamente se pueda calcular el costo de un día de tomas como la union de los conjuntos de esas tomas.

Empezamos con la matriz (como se recibe) y **vimos la necesidad de cambiar** a lista de sets porque la funcion objetivo es esencialmente una union de conjuntos o la sumatoria de costo de los actores que graban cada día y la matriz inicial obligaría a recorrer columnas repetidamente, mientras que tener la información por sets expresa la operacion de forma más natural y eficiente. La matriz solo se usa para cargar; despues se trabaja con los conjuntos.

B) Datos de la solución: cuáles tomas van cada día. Una planificacion de tomas se puede representar como una **lista de días**, y cada día como una **lista de indices de toma** (p.ej. `[[0,1,2], [3,4,5], ...]`). Con ello el algoritmo podría construir y/o modificar la solución (mover/intercambiar tomas en la búsqueda local), y permite comprobar la restriccion de ≤ 6 tomas/día con `len(día)`.

Conclusion: matriz solo como formato de carga → **lista de sets** para los datos de entrada (operacion clave = union de conjuntos), y **lista de listas** (días → tomas) para la solucion que se explora. Es la combinacion que mejor se adapta a la funcion objetivo y a las restricciones.

```

In [3]: # =====
# ESTRUCTURA DE DATOS - demostracion
# =====
# A) Entrada: matriz (carga) -> lista de SETS de actores por toma

ejemplo_matriz = matriz[0]                # fila de la toma 1
ejemplo_set     = tomas[0]                # {1,2,3,4,5}

print("Toma 1 como fila de matriz :", ejemplo_matriz)
print("Toma 1 como set de actores :", sorted(ejemplo_set))

# Operacion clave (coste de un día) = tamaño de la UNION de sets

dia_ejemplo = [0, 1, 5]                  # hago las tomas 1, 2, 6

union_actores = set().union(*[tomas[t] for t in dia_ejemplo])
print("\nDía grabando las tomas 1,2,6 -> necesito a los actores:", sorted(union_actores),
      "\n-> coste día (suponiendo $1 actor/día= $", len(union_actores))

# B) Solucion: lista de días, cada día una lista de indices de toma
solucion_ejemplo = [[0,1,2,3,4,5],[6,7,8,9,10,11]]

```

```
print("\nSolucion (lista de dias -> tomas):", solucion_ejemplo)
print("Tomas en el dia 1:", len(solucion_ejemplo[0]), "(<= 6 OK)")
```

Toma 1 como fila de matriz : [1, 1, 1, 1, 1, 0, 0, 0, 0, 0]
Toma 1 como set de actores : [1, 2, 3, 4, 5]

Dia grabando las tomas 1,2,6 -> necesito a los actores: [1, 2, 3, 4, 5] -> coste dia (suponiendo \$1 actor/día= \$ 5

Solucion (lista de dias -> tomas): [[0, 1, 2, 3, 4, 5], [6, 7, 8, 9, 10, 11]]
Tomas en el dia 1: 6 (<= 6 OK)

Según el modelo para el espacio de soluciones

(*)¿Cual es la función objetivo?

(*)¿Es un problema de maximización o minimización?

Respuesta:

Estamos ante un problema de **minimización**. La función objetivo es reducir al máximo el gasto total en sueldos/jornadas de los actores de doblaje. Cada actor cobra una tarifa fija por día, sin importar cuántas tomas haga. Por lo tanto, el costo de un día va a depender del número de actores distintos que participan en las tomas de ese día. Si agrupamos tomas que comparten actores en el mismo día, reducimos el número de días en que esos actores deben desplazarse.

De forma matemática definimos la siguiente variable binaria de decisión:

$y_{i,k} \in \{0,1\}$: Indica si el actor i tiene que ir a grabar el día k .

$y_{i,k} = 1$ si el actor i asiste a grabar el día k .

$y_{i,k} = 0$ si el actor i no asiste el día k .

Donde:

$i \in \{1,2,\dots,10\}$ (los 10 actores).

$k \in \{1,2,\dots,D\}$ (los días de grabación, donde D es entre 5 y 30 días).

Como todos los actores cobran exactamente la misma tarifa por día de desplazamiento (C), la función objetivo formal se expresa así:

$$\min \quad Z = C \cdot \sum_{i=1}^{10} \sum_{k=1}^D y_{i,k}$$

Peor escenario posible: Que los actores tengan que ir muchos días a grabar pocas tomas cada uno (por ejemplo, sumar 35 convocatorias de actor a lo largo del rodaje).

Escenario óptimo (lo que busca la función): Agrupar las 30 tomas de tal forma que los actores compartan días tanto como sea posible, reduciendo la suma total de asistencias (por ejemplo, lograr realizar todo con solo 18 convocatorias de actor en total).

Diseña un algoritmo para resolver el problema por fuerza bruta

Respuesta

```
In [4]: # --- Verificaciones rapidas ---
print("Nº tomas de prueba :", N_TOMAS)
print("Nº actores :", N_ACTORES)
print()
print("Ejemplos (toma -> actores):")
for t in [0,1,3]:
    print(f"  toma {t+1:2d}: {sorted(tomas[t])}")
print()
# Cuantas tomas hace cada actor
por_actor = {}
for a in range(1, N_ACTORES+1):
    contador = 0
    for f in matriz:
        if f[a-1]==1:
            contador +=1
    por_actor[a]= contador

print("Tomas por actor (si se tomaran las 30 tomas):", por_actor)

def calcular_coste(dias): #funcion para calcular el costo total
    """
    Recibe la estructura de 'dias' y suma cuántos actores ÚNICOS van cada día.
    Si el Actor 1 graba 3 tomas en el Día 1, solo se le cuenta 1 asistencia.
    """
    coste_total = 0
```

```

for dia in dias:
    # Usamos un conjunto para almacenar los actores que asisten en este día sin repetir
    actores_del_dia = set()

    # Cada 'dia' contiene una lista de tomas asignadas
    for toma_actores in dia:
        for actor in toma_actores:
            actores_del_dia.add(actor)

    # La cantidad de actores únicos asistiendo hoy es la longitud del conjunto
    coste_total += len(actores_del_dia)

return coste_total

# ALGORITMO RECURSIVO DE FUERZA BRUTA (BACKTRACKING)

def buscar_optimo(toma_idx=0, dias=()):
    """
    Evalúa todas las combinaciones posibles asignando una toma a la vez.
    - toma_idx: Número de la toma actual que estamos intentando colocar (0 a 29).
    - dias: Estado actual de la agenda de días con las tomas asignadas.
    """

    # CASO BASE: si ya se han asigado todas las tomas se calcula el costo
    if toma_idx == N_TOMAS:
        coste = calcular_coste(dias)
        return coste, dias

    # PASO RECURSIVO: Obtenemos el conjunto de actores de la toma actual
    actores_toma_actual = tomas[toma_idx]

    # se guardan los resultados de explorar cada camino posible
    opciones = []

    # -----
    # OPCIÓN A: Intentar meter la toma en un día que ya existe
    # -----
    for i in range(len(dias)):
        dia_actual = dias[i]

        # Restricción fundamental: Un día no puede tener más de 6 tomas
        if len(dia_actual) < 6:
            # se agrega la toma actual a la lista de tomas de ese día
            nuevo_dia = dia_actual + (actores_toma_actual,)

            # Reconstruimos la estructura de 'dias' reemplazando solo el día 'i'
            nuevos_dias = []
            for j in range(len(dias)):
                if j == i:
                    nuevos_dias.append(nuevo_dia)
                else:
                    nuevos_dias.append(dias[j])

            # Avanzamos a la siguiente toma (toma_idx + 1) con la nueva agenda
            resultado = buscar_optimo(toma_idx + 1, tuple(nuevos_dias))
            opciones.append(resultado)

    # -----
    # OPCIÓN B: Crear un nuevo día exclusivamente para colocar la toma
    # -----
    nuevo_dia_unico = (actores_toma_actual,)
    dias_con_nuevo = dias + (nuevo_dia_unico,)

    # Avanzamos a la siguiente toma (toma_idx + 1) habiendo creado un día más
    resultado_nuevo = buscar_optimo(toma_idx + 1, dias_con_nuevo)
    opciones.append(resultado_nuevo)

    # SELECCIÓN: Elegir la opción que produjo el coste más bajo
    # -----
    mejor_opcion = opciones[0]

    for op in opciones:
        coste_opcion = op[0]
        coste_mejor = mejor_opcion[0]

        # Si esta opción requiere menos asistencias, pasa a ser la nueva mejor
        if coste_opcion < coste_mejor:
            mejor_opcion = op

    return mejor_opcion

# 4. PRUEBA Y EJECUCIÓN
# =====

coste_minimo, plan_dias = buscar_optimo(0, ())

```

```
print(f"=== RESULTADOS PARA {N_TOMAS} TOMAS DE PRUEBA ===")
print("Mínimo de asistencias necesarias:", coste_minimo)
print("Número de días de grabación utilizados:", len(plan_dias))

# Desglose claro por cada día programado
print("\nDesglose de la agenda elegida:")
for num, dia in enumerate(plan_dias, start=1):
    actores_unicos = set()
    for toma in dia:
        for actor in toma:
            actores_unicos.add(actor)
    print(f"  Día {num}: {len(dia)} tomas programadas -> Actores convocados: {sorted(list(actores_unicos))}")
```

Nº tomas de prueba : 10
Nº actores : 10

Ejemplos (toma -> actores):
toma 1: [1, 2, 3, 4, 5]
toma 2: [3, 4, 5]
toma 4: [1, 2, 7, 8]

Tomas por actor (si se tomaran las 30 tomas): {1: 22, 2: 14, 3: 13, 4: 15, 5: 11, 6: 8, 7: 3, 8: 4, 9: 2, 10: 2}
=== RESULTADOS PARA 10 TOMAS DE PRUEBA ===
Mínimo de asistencias necesarias: 13
Número de días de grabación utilizados: 2

Desglose de la agenda elegida:
Día 1: 6 tomas programadas -> Actores convocados: [1, 2, 3, 4, 5, 7, 8]
Día 2: 4 tomas programadas -> Actores convocados: [1, 2, 4, 5, 6, 9]

Calcula la complejidad del algoritmo por fuerza bruta

Respuesta

La complejidad de la fuerza bruta es el producto de dos factores:

(1) Nº de candidatos a evaluar. Cada planificacion valida es una particion de las 30 tomas en días de <= 6 tomas. Como 30 = 5 × 6 y usar mas de 5 días nunca mejora el coste (cada día extra añade desplazamientos), basta explorar las particiones en **5 días de 6 tomas**, cuyo numero es:

$$\frac{30!}{(6!)^5 \cdot 5!} = 11\,423\,951\,396\,577\,720 \approx 1.14 \times 10^{16}$$

Este numero crece de forma **super-exponencial** con n (del orden del numero de Bell / Stirling): añadir tomas multiplica el espacio de busqueda por un factor creciente, por lo que es el termino que domina la complejidad.

$$\mathcal{O}(B_{30,\leq 6} \cdot N \cdot A) \approx \mathcal{O}(2^N)$$

(2) Coste de evaluar un candidato. La funcion objetivo recorre las n tomas una vez y, por cada una, une su conjunto de actores (hasta A = 10). Es decir, **O(n · A)** = O(30 · 10) ≈ 300 operaciones por candidato. Este factor es lineal y despreciable frente al (1).

Complejidad total:

$$O\left(\frac{n!}{(6!)^{n/6} (n/6)!} \cdot n \cdot A\right) \approx O(\text{super-exponencial})$$

El termino combinatorio (nº de particiones) domina; la evaluacion O(n·A) no cambia la clase de complejidad.

En tiempo real (con restriccion). Suponiendo 10⁹ operaciones/segundo: 1.14 × 10¹⁶ candidatos × 300 op ≈ 3.4 × 10¹⁸ operaciones ≈ **~109 años**.

Conclusion: aun con la restriccion de 6 tomas/día, la fuerza bruta es **intratable** para n = 30. Usar sets o código más rápido solo reduce la constante (las ~300 op), pero NO el termino super-exponencial: hace falta cambiar de estrategia (heuristica / metaheuristica) para tener un resultado mejor.

```
In [5]: # =====
# COMPLEJIDAD FUERZA BRUTA - estimacion de tiempo (con restriccion)
# =====
import math
N_TOMAS_TOTAL = len(matriz)

n, A, max_dia = N_TOMAS_TOTAL, N_ACTORES, MAX_TOMAS_POR_DIA # 30, 10, 6
n_dias = n // max_dia # 5 dias de 6 tomas

# Nº de candidatos con la restriccion (5 dias x 6 tomas):
candidatos = math.factorial(n) // (math.factorial(max_dia)**n_dias
                                * math.factorial(n_dias))

ops_por_candidato = n * A # coste O(n·A)
operaciones = candidatos * ops_por_candidato

OPS_POR_SEG = 1e9 # 1000 millones op/s
segundos = operaciones / OPS_POR_SEG
```

```

anhos = segundos / (3600 * 24 * 365)

print(f"Candidatos (5 dias x 6 tomas): {candidatos:.3e}")
print(f"Operaciones por candidato 0(n·A): {ops_por_candidato}")
print(f"Operaciones totales: {operaciones:.3e}")
print(f"Tiempo estimado a 1e9 op/s: {segundos:.3e} s ~ {anhos:,.0f} años")

```

Candidatos (5 dias x 6 tomas): 1.142e+16
Operaciones por candidato 0(n·A): 300
Operaciones totales: 3.427e+18
Tiempo estimado a 1e9 op/s: 3.427e+09 s ~ 109 años

(*)Diseña un algoritmo que mejore la complejidad del algortimo por fuerza bruta. Argumenta porque crees que mejora el algoritmo por fuerza bruta

Respuesta

Como el problema es de tipo combinatorio (particion de conjuntos, NP-duro), la fuerza bruta es intratable. Proponemos un metodo **heuristico** en dos etapas que encuentra soluciones muy buenas en tiempo polinomico:

Etapas 1 - Greedy constructivo (clustering). Cada toma empieza en su propio dia y se fusionan repetidamente los dos grupos que menos actores nuevos añaden (los que mas comparten), respetando el limite de 6 tomas/dia, hasta quedar 5 dias. Aplicando directamente la intuicion del problema: juntar tomas que comparten actores. Coste: **O(n³·A)**.

Etapas 2 - Busqueda local (refinamiento). Sobre la solucion greedy se aplican pequeños cambios (Mover una toma a otro dia, o Intercambiar dos tomas entre dias) y se aceptan si no empeoran el coste (hill-climbing). Con reinicios aleatorios se evitan optimos locales. Cada iteracion es O(n·A).

Por qué mejora a la fuerza bruta?. La fuerza bruta es super-exponencial (numero de Bell ~10¹⁶–10²³) y tarda siglos. La heuristica NO enumera todas las particiones: construye una y la refina, con complejidad **polinomica**, y termina en milisegundos. El precio es que no garantiza el optimo (aunque puede alcanzarlo):

```

In [6]: import itertools, random

# =====
# ALGORITMO MEJORADO (HEURISTICO) – Problema 1: Sesiones de doblaje
# Dos etapas: (1) Greedy constructivo -> (2) Busqueda local
# =====
def coste_dia(dia):
    """Actores distintos necesarios en un dia (union de sus tomas)."""
    actores = set()
    for t in dia:
        actores |= tomas[t]
    return len(actores)

def coste(planificacion):
    """Funcion objetivo: total de actor-dias de una planificacion."""
    return sum(coste_dia(dia) for dia in planificacion)

def es_valida(planificacion):
    """Restricciones del problema:
    - ningun dia con mas de 6 tomas
    - las 30 tomas aparecen exactamente una vez."""
    todas = [t for dia in planificacion for t in dia]
    if any(len(dia) > MAX_TOMAS_POR_DIA for dia in planificacion):
        return False
    return sorted(todas) == list(range(N_TOMAS_TOTAL))

# ---- ETAPA 1: GREEDY CONSTRUCTIVO (clustering por solapamiento) ----
# Cada toma empieza en su propio dia; fusionamos repetidamente los
# dos grupos cuya union añade menos actores nuevos (mas comparten),
# sin superar 6 tomas/dia, hasta quedarnos con 5 dias.
def greedy_constructivo(ndias=5):
    grupos = [[t] for t in range(N_TOMAS_TOTAL)]
    while len(grupos) > ndias:
        mejor = None # (delta, i, j)
        for i, j in itertools.combinations(range(len(grupos)), 2):
            if len(grupos[i]) + len(grupos[j]) <= MAX_TOMAS_POR_DIA:
                fusion = grupos[i] + grupos[j]
                delta = coste_dia(fusion) - coste_dia(grupos[i]) - coste_dia(grupos[j])
                if mejor is None or delta < mejor[0]:
                    mejor = (delta, i, j)
        if mejor is None:
            break
        _, i, j = mejor
        grupos[i] = grupos[i] + grupos[j]
        del grupos[j]
    return grupos

# ---- ETAPA 2: BUSQUEDA LOCAL (refinamiento, hill-climbing) ----
# Pequeños cambios sobre la solucion: MOVER una toma a otro dia o
# INTERCAMBIAR dos tomas entre dias. Se acepta si no empeora el
# coste. Con varios reinicios escapamos de optimos locales.

```

```
def busqueda_local(dias, iters=20000, seed=0):
    random.seed(seed)
    dias = [d[:] for d in dias]
    best = coste(dias)
    for _ in range(iters):
        if random.random() < 0.5: # MOVER
            d1, d2 = random.sample(range(len(dias)), 2)
            if not dias[d1] or len(dias[d2]) >= MAX_TOMAS_POR_DIA:
                continue
            i = random.randrange(len(dias[d1]))
            t = dias[d1].pop(i); dias[d2].append(t)
            c = coste(dias)
            if c <= best: best = c
            else: dias[d2].pop(); dias[d1].insert(i, t) # deshacer
        else: # INTERCAMBIAR
            d1, d2 = random.sample(range(len(dias)), 2)
            if not dias[d1] or not dias[d2]:
                continue
            i = random.randrange(len(dias[d1])); j = random.randrange(len(dias[d2]))
            dias[d1][i], dias[d2][j] = dias[d2][j], dias[d1][i]
            c = coste(dias)
            if c <= best: best = c
            else: dias[d1][i], dias[d2][j] = dias[d2][j], dias[d1][i] # deshacer
    return dias, best

# ---- HEURISTICA COMPLETA ----
def resolver(reinicios=10):
    inicial = greedy_constructivo()
    mejor_sol, mejor_c = inicial, coste(inicial)
    for s in range(reinicios):
        sol, c = busqueda_local(inicial, seed=s)
        if c < mejor_c:
            mejor_c, mejor_sol = c, sol
    return mejor_sol, mejor_c

sol, c = resolver()
print("Etapa 1 (greedy)      :", coste(greedy_constructivo()), "actor-dias")
print("Etapa 2 (busqueda local):", c, "actor-dias  valida?", es_valida(sol))

for k, d in enumerate(sol, 1):
    ac = sorted(set().union(*[tomas[t] for t in d]))
    print(f" Dia {k}: tomas {sorted(t+1 for t in d)} -> {len(ac)} actores {ac}")
```

Etapa 1 (greedy) : 31 actor-dias
Etapa 2 (busqueda local): 27 actor-dias valida? True
Dia 1: tomas [1, 5, 9, 11, 20, 30] -> 6 actores [1, 2, 3, 4, 5, 8]
Dia 2: tomas [7, 13, 16, 25, 27, 28] -> 5 actores [1, 2, 4, 5, 10]
Dia 3: tomas [3, 4, 8, 15, 21, 29] -> 6 actores [1, 2, 5, 6, 7, 8]
Dia 4: tomas [2, 6, 10, 12, 22, 26] -> 7 actores [1, 2, 3, 4, 5, 6, 9]
Dia 5: tomas [14, 17, 18, 19, 23, 24] -> 3 actores [1, 3, 6]

(*)Calcula la complejidad del algoritmo

Respuesta

Complejidad del algoritmo mejorado (heuristico), en dos etapas:

Etapa 1 - Greedy constructivo. Realiza ~(*n* – 5) fusiones. En cada fusion examina todos los pares de grupos, *O*(*n*²), y evalua cada union en *O*(*A*). Coste:

$$O(n^3 \cdot A)$$

Etapa 2 - Busqueda local. Con **I** iteraciones; en cada una hace un movimiento (mover/intercambiar) y recalcula el coste en *O*(*n* · *A*). Coste:

$$O(I \cdot n \cdot A)$$

Total: polinomico, dominado por *O*(*n*³·*A*) del greedy (*I* es una constante fijada). Frente a la fuerza bruta (super exponencial, numero de Bell ~10¹⁶–10²³) o a los metodos exactos (DP *O*(2^{*n*}), ILP exponencial), la heuristica es **polinomica** y termina en milisegundos, a cambio de no garantizar el optimo (aunque en la practica puede alcanzarlo).

```
In [7]: # =====
# COMPLEJIDAD DEL ALGORITMO MEJORADO - comprobacion empirica
# =====
# Medimos el tiempo del greedy al crecer n para ver el comportamiento
# polinomico (muy lejos del super-exponencial de la fuerza bruta).
# Generador minimo e INLINE (no depende de celdas posteriores).
import time, math, random
_orig = (matriz, tomas, N_TOMAS_TOTAL, N_ACTORES, MAX_TOMAS_POR_DIA)

print(" n | tiempo greedy (s)")
for nt in [30, 60, 120, 240]:
    rng = random.Random(7)
    tks = [set(rng.sample(range(1, 13), rng.randint(1, 5))) for _ in range(nt)]
```

```

tomas = tks
N_TOMAS_TOTAL, N_ACTORES, MAX_TOMAS_POR_DIA = nt, 12, 6
t0 = time.time(); greedy_constructivo(math.ceil(nt/6)); dt = time.time()-t0
print(f" {nt:3d} | {dt:.3f}")

matriz, tomas, N_TOMAS_TOTAL, N_ACTORES, MAX_TOMAS_POR_DIA = _orig # restaurar
print("(crecimiento polinomico 0(n^3*A), no exponencial)")

n | tiempo greedy (s)
30 | 0.007
60 | 0.031
120 | 0.143
240 | 1.035
(crecimiento polinomico 0(n^3*A), no exponencial)

```

Enfoque alternativo (EXACTO): Programación Lineal Entera (ILP)

Ademas de la heuristica, el problema se puede modelar como un **problema de Programacion Lineal Entera** (variables binarias 0/1), analogo a uno de los problemas vistos en clase. Un solver (CBC) devuelve el **optimo demostrado**.

Fijamos **D = 5 dias** (= 30/6, el minimo posible). Esta eleccion no pierde el optimo: se ha comprobado resolviendo el ILP con D = 6...13 dias (y con ≥ 14 dias el coste es $\geq 2 \cdot 14 = 28 > 27$, pues cada día no vacio requiere ≥ 2 actores): ninguna planificacion con mas de 5 dias baja de 27.

Variables: $x[t,d]=1$ si la toma t se graba el dia d; $y[a,d]=1$ si el actor a debe acudir el dia d.

Objetivo: minimizar los actor-dias $\rightarrow \min \sum y[a,d]$

Restricciones:

1. cada toma en exactamente un dia: $\sum_d x[t,d] = 1$
2. maximo 6 tomas por dia: $\sum_t x[t,d] \leq 6$
3. enlace: si la toma t (con actor a) va el dia d, el actor acude: $y[a,d] \geq x[t,d]$

A continuación el desarrollo:

In [8]: `!pip install pulp`

Requirement already satisfied: pulp in /Users/bhuvanacheemala/MIAR/.venv/lib/python3.13/site-packages (3.3.2)

[notice] A new release of pip is available: 26.1 -> 26.1.2

[notice] To update, run: `pip install --upgrade pip`

In [9]:

```

# =====
# ENFOQUE ALTERNATIVO (EXACTO): PROGRAMACION LINEAL ENTERA (ILP)
# Devuelve el OPTIMO DEMOSTRADO usando el solver CBC (via PuLP).
# =====
# En Google Colab PuLP viene instalado; si no: !pip install pulp

import pulp, time

D = 5 # 5 dias x 6 tomas = 30

prob = pulp.LpProblem("doblaje", pulp.LpMinimize)
# x[t,d]=1 si la toma t se graba el dia d ; y[a,d]=1 si el actor a acude el dia d
x = {(t,d): pulp.LpVariable(f"x_{t}_{d}", cat="Binary")
      for t in range(N_TOMAS_TOTAL) for d in range(D)}
y = {(a,d): pulp.LpVariable(f"y_{a}_{d}", cat="Binary")
      for a in range(N_ACTORES) for d in range(D)}

# Objetivo: minimizar actor-dias
prob += pulp.lpSum(y[a,d] for a in range(N_ACTORES) for d in range(D))

# (1) cada toma en exactamente un dia
for t in range(N_TOMAS_TOTAL):
    prob += pulp.lpSum(x[t,d] for d in range(D)) == 1
# (2) maximo 6 tomas por dia
for d in range(D):
    prob += pulp.lpSum(x[t,d] for t in range(N_TOMAS_TOTAL)) <= MAX_TOMAS_POR_DIA
# (3) enlace actor-dia: si la toma t (con actor a) va el dia d, el actor acude
for t in range(N_TOMAS_TOTAL):
    for a in range(N_ACTORES):
        if matriz[t][a] == 1:
            for d in range(D):
                prob += y[a,d] >= x[t,d]

# Rompe-simetria: ordena los dias por su toma de menor indice
# (evita explorar los 5! ordenes equivalentes de dias -> mucho mas rapido)
for d in range(1, D):
    for t in range(N_TOMAS_TOTAL):
        prob += x[t,d] <= pulp.lpSum(x[tp,d-1] for tp in range(t))

t0 = time.time()
prob.solve(pulp.PULP_CBC_CMD(msg=0, timeLimit=60))
print("Estado del solver:", pulp.LpStatus[prob.status])

```

```
print("OPTIMO DEMOSTRADO =", int(round(pulp.value(prob.objective))), "actor-dias")
print("Tiempo:", round(time.time()-t0, 1), "s")
for d in range(D):
    tk = sorted(t+1 for t in range(N_TOMAS_TOTAL) if pulp.value(x[t,d]) > 0.5)
    ac = sorted(a+1 for a in range(N_ACTORES) if pulp.value(y[a,d]) > 0.5)
    print(f" Dia {d+1}: tomas {tk} -> {len(ac)} actores {ac}")
```

Estado del solver: Optimal
OPTIMO DEMOSTRADO = 27 actor-dias
Tiempo: 31.3 s

Dia 1: tomas [1, 2, 5, 11, 20, 26] -> 7 actores [1, 2, 3, 4, 5, 8, 9]
Dia 2: tomas [3, 4, 10, 15, 21, 29] -> 7 actores [1, 2, 5, 6, 7, 8, 9]
Dia 3: tomas [6, 7, 13, 27, 28, 30] -> 4 actores [1, 2, 4, 5]
Dia 4: tomas [8, 9, 12, 16, 22, 25] -> 6 actores [1, 2, 3, 4, 6, 10]
Dia 5: tomas [14, 17, 18, 19, 23, 24] -> 3 actores [1, 3, 6]

Comparativa de los dos enfoques (ambos alcanzan 27 actor-dias):

Enfoque	Coste	¿Garantiza optimo?	Complejidad	Tiempo	Escala a n grande
Heuristico (greedy + busqueda local)	27	No (pero lo alcanza)	Polinomial $O(n^3 \cdot A)$	milisegundos	Si
ILP exacto (CBC)	27	Si (lo demuestra)	Exponencial (NP-duro)	~segundos	No

Justificacion: el ILP **demuestra** que 27 es el minimo, luego confirma que la heuristica encuentra el optimo real. Como ambos dan el mismo coste (27), se elige la **heuristica** como solucion practica (mismo coste, mucho mas rapida y escalable), usando el ILP como validacion de optimalidad.

Según el problema (y tenga sentido), diseña un juego de datos de entrada aleatorios

Respuesta

Un juego de datos de entrada del problema es una **matriz de participacion actorxtoma** (0/1). El generador `generar_datos` crea instancias aleatorias **realistas** y parametrizables:

- N° de tomas y de actores configurables (por defecto 30 y 10, como el problema).
- Cada toma tiene entre `min_act` y `max_act` actores, y **nunca** queda vacia.
- Los actores tienen **popularidad desigual** (unos participan en muchas mas tomas que otros), igual que en los datos reales, donde hay protagonistas que aparecen casi siempre y secundarios con pocas tomas.
- `seed` para reproducibilidad.

Esto permite **probar el algoritmo con distintos tamaños y densidades** y comprobar que sigue funcionando (validez de la solucion y coste razonable) mas alla del juego de datos concreto del enunciado.

```
In [10]: # =====
# JUEGO DE DATOS ALEATORIO - Problema 1: Sesiones de doblaje
# =====
import random

def generar_datos(n_tomas=30, n_actores=10, min_act=1, max_act=5,
                  max_por_dia=6, seed=None):
    """Genera un problema aleatorio realista:
    - cada toma tiene entre min_act y max_act actores (>=1, nunca vacia)
    - los actores tienen POPULARIDAD distinta (unos salen mas que otros),
      como en los datos reales (p.ej. un protagonista en casi todas).
    Devuelve (matriz, tomas_sets, max_por_dia)."""
    rng = random.Random(seed)
    # peso de popularidad por actor (decreciente) -> reparto desigual realista
    pesos = [n_actores - a for a in range(n_actores)] # actor 1 el mas popular
    matriz, tomas = [], []
    for _ in range(n_tomas):
        k = rng.randint(min_act, max_act) # nº actores en la toma
        elegidos = set(rng.choices(range(n_actores), weights=pesos, k=k))
        if not elegidos: # garantizar no vacia
            elegidos = {rng.randrange(n_actores)}
        fila = [1 if a in elegidos else 0 for a in range(n_actores)]
        matriz.append(fila)
        tomas.append({a+1 for a in elegidos})
    return matriz, tomas, max_por_dia

# --- Generar un juego de ejemplo (mismo tamaño que el problema) ---
m_rnd, tomas_rnd, maxd = generar_datos(n_tomas=30, n_actores=10, seed=42)
print("Juego aleatorio generado: 30 tomas, 10 actores, max 6/dia (seed=42)")
print("Actores por toma (primeras 10):", [len(t) for t in tomas_rnd[:10]])
part = sum(len(t) for t in tomas_rnd)
print("Participaciones totales:", part, "| media actores/toma:", round(part/30,2))
por_actor = {a: sum(1 for t in tomas_rnd if a in t) for a in range(1,11)}
print("Tomas por actor:", por_actor)
```

Juego aleatorio generado: 30 tomas, 10 actores, max 6/dia (seed=42)
Actores por toma (primeras 10): [1, 2, 4, 1, 4, 2, 2, 1, 1, 2]
Participaciones totales: 71 | media actores/toma: 2.37
Tomas por actor: {1: 12, 2: 12, 3: 8, 4: 10, 5: 8, 6: 8, 7: 6, 8: 2, 9: 2, 10: 3}

Aplica el algoritmo al juego de datos generado

Respuesta

Aplicamos la heurística (greedy + búsqueda local) a instancias generadas por `generar_datos` . Como las funciones usan las globales del problema, la funcion `cargar_instancia` las sustituye por la instancia aleatoria antes de resolver (y al final se restauran los datos originales).

Resultados: en todos los casos la solucion es **valida** (≤6 tomas/dia y todas las tomas asignadas) y el coste queda por encima pero cerca de la cota inferior teorica, tanto en la instancia de ejemplo como al variar tamaño y densidad (18×8, 30×10, 48×12, 60×15). Esto confirma que el algoritmo **generaliza** mas alla del juego de datos del enunciado.

```
In [11]: # =====
# APLICAR LA HEURISTICA AL JUEGO DE DATOS ALEATORIO
# =====
"""
Las funciones (coste, greedy_constructivo, resolver...) usan las variables globales del problema. Para resolver una
instancia
aleatoria, cargamos sus datos en esas globales y llamamos a resolver().
"""
import math, copy

# Guardar los datos ORIGINALES del problema para restaurarlos al final
_orig = (matriz, tomas, N_TOMAS_TOTAL, N_ACTORES, MAX_TOMAS_POR_DIA)

def cargar_instancia(m, tks, maxd):
    global matriz, tomas, N_TOMAS_TOTAL, N_ACTORES, MAX_TOMAS_POR_DIA
    matriz, tomas = m, tks
    N_TOMAS_TOTAL, N_ACTORES = len(m), len(m[0])
    MAX_TOMAS_POR_DIA = maxd

def resolver_instancia(n_tomas, n_actores, seed):
    m, tks, maxd = generar_datos(n_tomas, n_actores, seed=seed)
    cargar_instancia(m, tks, maxd)
    sol, c = resolver(reinicios=10) # greedy + búsqueda local
    lb = sum(math.ceil(sum(1 for f in m if f[a]==1)/maxd) for a in range(n_actores))
    return sol, c, lb

# --- 1) Resolver la instancia de ejemplo (seed=42, 30 tomas) ---
sol, c, lb = resolver_instancia(30, 10, seed=42)
print("Instancia aleatoria seed=42 (30 tomas, 10 actores):")
print(f" Coste heuristica = {c} actor-dias | cota inferior = {lb} | valida = {es_valida(sol)}")
for k, d in enumerate(sol, 1):
    ac = sorted(set().union(*[tomas[t] for t in d])) if d else []
    print(f" Dia {k}: tomas {sorted(t+1 for t in d)} -> {len(ac)} actores {ac}")

# --- 2) Probar con varios tamaños y semillas (robustez) ---
print("\nPruebas con distintos tamaños/semillas:")
print(" tomas x actores | seed | coste | cota inf | valida")
for (nt, na, sd) in [(18,8,1),(30,10,2),(48,12,3),(60,15,4)]:
    sol, c, lb = resolver_instancia(nt, na, sd)
    print(f" {nt:2d} x {na:2d} | {sd} | {c:3d} | {lb:3d} | {es_valida(sol)}")

# --- Restaurar los datos ORIGINALES del problema ---
matriz, tomas, N_TOMAS_TOTAL, N_ACTORES, MAX_TOMAS_POR_DIA = _orig
print("\n(datos originales del problema restaurados)")
```

Instancia aleatoria seed=42 (30 tomas, 10 actores):

Coste heuristica = 25 actor-dias | cota inferior = 16 | valida = True

Dia 1: tomas [1, 8, 16, 27, 28, 30] -> 4 actores [1, 5, 7, 9]

Dia 2: tomas [2, 4, 7, 10, 23, 29] -> 4 actores [1, 2, 4, 6]

Dia 3: tomas [3, 15, 18, 24, 25, 26] -> 6 actores [1, 2, 3, 4, 5, 10]

Dia 4: tomas [6, 11, 12, 17, 21, 22] -> 6 actores [1, 2, 4, 5, 7, 8]

Dia 5: tomas [5, 9, 13, 14, 19, 20] -> 5 actores [2, 3, 6, 7, 10]

Pruebas con distintos tamaños/semillas:

tomas x actores	seed	coste	cota inf	valida
18 x 8	1	17	13	True
30 x 10	2	25	18	True
48 x 12	3	42	26	True
60 x 15	4	61	35	True

(datos originales del problema restaurados)

Enumera las referencias que has utilizado(si ha sido necesario) para llevar a cabo el trabajo

Respuesta

Bibliografía de la asignatura

- Brassard, G. y Bratley, P. (1997). *Fundamentos de algoritmia*. Prentice Hall. ISBN 9788489660007.
- Guerequeta, R. y Vallecillo, A. (2000). *Tecnicas de diseño de algoritmos*. Universidad de Malaga.

- Lee, R. C. T., Tseng, S. S., Chang, R. C. y Tsai, Y. T. (2005). *Introduccion al diseño y analisis de algoritmos*. ISBN 9789701061244.
- Duarte, A. (2008). *Metaheurísticas*. Madrid: Dykinson.
- Material y videoconferencias de la asignatura *Algoritmos de Optimizacion* (VIU).

Técnicas y conceptos

- Numeros de Bell / particiones de conjuntos y numeros de Stirling de segunda especie (para el conteo de posibilidades).
- Heurísticas constructivas (greedy / clustering aglomerativo) y búsqueda local (hill-climbing) como metaheurística de mejora.
- Ramificación y poda (branch & bound) y programación dinámica sobre subconjuntos como métodos exactos.
- Programación Lineal Entera (formulación con variables binarias 0/1), analoga al problema de inversión de capital visto en clase.

Herramientas y librerías

- PuLP (modelado de PL entera) con el solver CBC (COIN-OR Branch and Cut).
- Documentación de PuLP: <https://coin-or.github.io/pulp/>

Describe brevemente las líneas de como crees que es posible avanzar en el estudio del problema. Ten en cuenta incluso posibles variaciones del problema y/o variaciones al alza del tamaño

Respuesta

1. Mejoras algorítmicas. La búsqueda local se puede reforzar con metaheurísticas más potentes: **recocido simulado** (acepta empeoramientos controlados para escapar de óptimos locales), **búsqueda tabu** (memoria de movimientos recientes), **algoritmos genéticos** (población de planificaciones que se cruzan y mutan) o **colonia de hormigas**. También se pueden usar vecindarios más ricos (mover bloques de tomas, reasignar días completos) y multiarranque para mayor robustez.

2. Métodos exactos mas escalables. Para certificar el óptimo en instancias mayores: mejores **cotas inferiores** para la ramificación y poda, formulaciones ILP más ajustadas, resolución con solvers comerciales (Gurobi, CPLEX) o técnicas matemáticas como la de **generación de columnas**, en donde solo se toman las variables necesarias para solucionar el problema en vez de tener que cargarlas todas desde el inicio dado que el problema es una partición de conjuntos.

3. Variaciones al alza del tamaño. Al crecer n , los métodos exactos (fuerza bruta, DP $O(2^n)$, ILP) dejan de ser viables, pero la **heurística polinómica $O(n^3 \cdot A)$ escala** y sigue dando buenas soluciones en milisegundos/segundos. Para tamaños muy grandes se puede **paralelizar** la búsqueda (varios arranques a la vez) y usar conjuntos para acelerar la evaluación del coste.

En general, si se hacen variaciones al problema como tener cobro distinto por actor o precedencias obligatorias entre tomas, entre otros; el problema sigue siendo minimizar coste y a la vez el número de días, por lo que en general, la estructura (partición de conjuntos con función objetivo de unión) se mantiene, y la mayoría de variaciones se modelan añadiendo restricciones o cambiando la función de coste, reutilizando el mismo esquema heurístico/exacto.

In []: